

从 Skill 1.0 到 Skill 2.0：模块化 Skill 中间件与投研场景应用

——以固定收益研究为例

From Skill 1.0 to Skill 2.0: A Modular Skill Middleware for Research Applications

叶青

2026年8月

摘要

：Skill 1.0 完成了研究能力的第一次工程化——"拉数→算指标→比阈值→写判断→出报告"的链条被逐一封装成可运行的 Skill，回答了"能否封装"。当体系膨胀到 13 个主题数据库、约 500 个分析逻辑、多本手册持续运行后，"如何封装得更好"成为第二次工程化的主题：封装粒度、激活上下文、接口契约、知识维护、复用方式与交付形态都需要重新设计。本文提出 Skill 2.0，以六个增量维度（D1~D6）刻画这一演进：封装单位从黑盒整体降为六类独立模块（D1），激活从全局全量改为按需指名（D2），接口从自然语言升级为 IN/OUT/DEP 数据契约（D3），知识从写死阈值升级为回验基线（D4），复用与维护从整体搬走升级为模块级引用与版本（D5），生产形态从教学文档升级为手册化中间件（D6）。全文以实践证据收束每个增量：资金面库 34 视角只取一模块时上下文占用降至约 1/30；一次"90%→70%"的回验纠错避免把缺数导致的极端值写成市场判断；架构手册拆解本身就是模块化的又一实例（图表均为真实操作复刻）。Skill 2.0 是对 Skill 1.0 的增量而非替代：两者共享"研究能力皆可工程化"的立场，不同之处在于封装与契约的粒度。

关键词：AI Agent、Skill 模块化、中间件、增量演进、投研数字化、固定收益

1. 引言

1.1 问题的演变

Skill 1.0 解决的问题是"研究能力能否封装"。以固定收益研究为例，数据拉取、指标计算、阈值比较、判断成文、报告输出的整条链条被拆成一层层 Skill，并在生产环境跑通。在此基础上，体系开始快速长大：**13 个主题数据库、约 500 个分析逻辑、多本技术手册**在生产运行，每个数据库由数十个分析视角堆叠而成——资金面库 34 个视角，信用利差库 11 个视角 × 5 路数据源，可转债库 22 个视角。

规模把"能否封装"的回答推向"如何封装更好"。当"整个 Skill"同时充当封装粒度和运行单元时，一批工程特性约束开始显形：

- **上下文膨胀**：激活即全量注入，与当前任务无关的内容常占大头；34 视角的库哪怕只用 1 个视角，也要背全套上下文。
- **复用过粗**：想复用"一个视角"往往要整体搬走整套数据库，跨库拷贝导致副本累积与版本漂移。
- **知识难更新**：阈值、公式写死在文档与代码里，市场中枢变化后更新成本高、更新动机弱。
- **集成成本高**：外部接入只能读文档、猜依赖，接口没有机器可校验的契约。

这些约束的本质是同一个：可复用、可运行的最小单元，不该是"整个 Skill"，而应是"一个模块"。

1.2 本文定位

本文是 Skill 系列的第二篇工作论文。v1 论文（research-skills 《Skill 中间件架构》，2026 年 7 月）已详述总体架构（四层架构、数据源可切换、组装层与回测引擎）与四条设计原则（ALL SKILL、聚焦产生深度、链条产生精确、分工产生价值），本文不重复这些内容，只回答一个问题：框架跑大之后，Skill 本身的工程形态如何从"整体可用的黑盒"进化为"模块化中间件"。

文章主线是六个增量维度 D1~D6（第 2 节），随后落到增量如何落地（第 3 节），再以真实案例与测量为证据（第 4 节），最后讨论与 v1 的关系、局限与迁移展望（第 5 节），结论收束（第 6 节）。

2. 六个增量维度

2.1 总览

六个增量维度构成全文骨架，总览如下（对应共享图表 fig4）：

维度	Skill 1.0	Skill 2.0 增量	实践价值 / 证据
D1 封装单位	一个目录一个 SKILL.md，整体交付	六类独立模块（SKILL.md / knowledge / templates / data_contract / scripts / config），独立交付、测试、版本化	最小可复用单元从"一个数据库"降为"一个视角 / 一个契约 / 一个引擎"；资金面 34 视角、信用利差 11×5、可转债 22 视角
D2 激活与上下文	全局注册，激活即全量注入上下文	默认不激活，按路径指名加载模块	Token 经济学：34 视角的库只取 1 个视角，上下文占用约从 100% 降到 1/30（fig7 真实测量）
D3 接口	触发条件用自然语言，无从机器校验	IN/OUT/DEP YAML 数据契约，可单测、可替换、可集成校验	集成成本从"读文档理解"降为"读契约就接"；换实现不扰外部（fig8 契约验

维度	Skill 1.0	Skill 2.0 增量	实践价值 / 证据
			收示例)
D4 知识	阈值/公式写死在文档与代码, 经验随人走	knowledge = 公式 + 阈值 + 案例 + 回验基线; 框架 > 数值; 市场回验驱动更新	"越用越深"从口号变机制 (fig9: 资金面分位 90%→70% 真实纠错)
D5 复用与维护	复用 = 整体搬走整套; 维护 = 整体换版	单模块被多库引用, 一处维护多处受益; 视角级升级与回退	卖方建组件、买方装配成为真实分工 (fig10; docs/skill-matrix.md 佐证生产运行)
D6 生产形态	交付一份"怎么用"的教学文档	介绍手册 (做什么/怎么用) 与架构手册 (模块注册表 = 契约清单/接口/独立调用示例) 分家	架构手册本身即 2.0 实例; 图表全部为真实操作复刻 (fig2/fig3/fig5, fioutput Navy 体系)

2.2 D1 封装单位：从黑盒整体到六类模块

Skill 1.0 的交付单元是一个目录加一份 SKILL.md, 内部结构对外不透明, 交付、测试、版本化都以整体为单位。Skill 2.0 规定标准结构由六类职责单一的模块组成:

```
skill-XXX/
├─ SKILL.md          ← 激活与路由 (触发条件 + 视角清单 + 模块索引)
├─ knowledge/        ← 知识模块 (公式 + 阈值 + 案例 + 回验基线)
├─ templates/        ← 输出模板模块 (表 / 图 / 文字形态)
├─ data_contract/    ← 数据契约模块 (字段定义, 与数据源解耦)
├─ scripts/          ← 可执行脚本模块 (取数 / 计算引擎, 自包含)
└─ config/           ← 配置模块 (参数 / 路由 / 版本)
```

关键变化不是目录变多, 而是**每个模块承载独立职责与独立契约, 可被单独调用、单独测试、单独版本化**。在现有体系中这一粒度已是现实: 资金面库的 34 个视角、信用利差库的 11×5 视角组合、可转债库的 22 个视角, 每个视角都可作为一个模块交付; 即便是以复杂著称的财务分析 (14 个 Agent 分 5 阶段协作), 其协作单元也被约束在统一契约之下, 不再依赖把整套能力一次性搬进上下文。最小可复用单元从"一个数据库"降为"一个视角 / 一个契约 / 一个引擎" (fig6)。实践价值: 复用、测试、升级都只触碰所需模块, 其余部分保持稳定。

2.3 D2 激活与上下文：从全局全量到按需指名

Skill 1.0 通过全局注册、语义匹配激活进入工作流, 激活一次即全量注入上下文, 与任务无关的部分成为持续噪声。Skill 2.0 放弃"全局可用"这一默认假设: **模块默认不激活, 调用方按路径指名加载**, SKILL.md 退化为可选的路由索引。

实践价值最直观的量化是 Token 经济学: 资金面库共 34 个视角, 一次"隔夜资金分位"判断只指名加载 `funding_fetcher` 这一个模块及其依赖, 其余 33 个视角不进上下文, 上下文占用从 100% 降到约 1/30 (fig7 生产测量; 换算成上下文预算即"128k→4k token 量级", 此为量级估算)。开销下降之外, 按需指名还有两个附带收益: 多 Skill 可在同一会话共存而不互相污

染；调用方的依赖面收敛到该模块的契约，与其它视角的演进彻底解耦（fig1，DASH 只取一模块的真实会话）。

2.4 D3 接口：从自然语言到 IN/OUT/DEP 契约

Skill 1.0 的触发条件以自然语言表达，外部调用方只能“读文档理解”，依赖关系无从机器校验。Skill 2.0 为每个模块声明 IN/OUT/DEP 三部分：**IN**（输入字段、类型、频度与缺数容忍）、**OUT**（输出结构与枚举）、**DEP**（运行时依赖的其它模块）。有了契约，模块即可单测、可替换、可集成校验——集成成本从“读文档理解”降为“读契约就接”，换实现不扰外部调用方（fig8 契约验收示例见 3.1 节）。

2.5 D4 知识：从写死阈值到回验基线

Skill 1.0 中，阈值、公式写死在文档与代码，经验的更新依赖人的自觉，市场中枢漂移后旧阈值继续服役。Skill 2.0 把知识层结构化为“公式 + 阈值 + 案例 + **回验基线**”四件套，并立两条运行纪律：**框架优先于数值**（口径、链路、计算先验正确，再谈数值是否可信）；**市场回验驱动更新**（取数 → 算分位 → 标注市场事件 → 对照基线找中枢漂移 → 更新阈值）。

实践价值是“越用越深”从口号变成机制。真实证据：某周资金面分位计算值落在 90% 以上的极端区，回验对照基线发现上游数据源缺失近三个交易日，数据补齐后回归 70% 区间（fig9）。若没有回验基线，“极端值”很可能会被直接写成市场判断进报告。

2.6 D5 复用与维护：从整体搬走到模块级引用与版本

Skill 1.0 的复用是“整体搬走整套”，维护是“整体换版”——跨库复用的拷贝散落多处，升级一次牵动全体，版本漂移随之而来。Skill 2.0 下，单模块可被多库**引用**而非复制：资金面分位模块同时被信用、宏观等数据库共用，一处维护、多处受益；升级与回退以视角为单位进行，牵动面控制在引用该模块的调用方。

实践价值是分工结构落地：**卖方建组件、买方装配**从一句口号变成真实的生产分工——组件在模块粒度上被双方共享，文档侧的模块注册表（docs/skill-matrix.md）在持续运行中记录“哪个模块被哪些库引用”（fig10）。跨库复用的维护成本从“每份拷贝改一遍”降为“改一处、验一处”。

2.7 D6 生产形态：从教学文档到手册化中间件

Skill 1.0 的交付物是一份“怎么用”的教学文档，使用者与集成者共用一份说明，使用说明与实现细节混写一处。Skill 2.0 将其拆为两类手册：**介绍手册**回答“做什么、怎么用”，面向使用者；**架构手册**回答“模块怎么组织、接口契约是什么、如何接入”，面向集成者，内含模块注册表、契约清单与独立调用示例。

实践价值有二。其一，**架构手册本身就是 Skill 2.0 的一个实例**：手册中的模块图对应真实目录结构，契约清单对应 data_contract 与 IN/OUT/DEP，集成者可拿手册对照代码逐项验收——文档即中间件的一部分，随代码演进同步更新。其二，手册中的图表不是示意画图，而是

DASH 对话实录、生产手册实拍与真实跑数输出的高保真复刻（3x DPI，fioutput Navy 体系，fig2/fig3/fig5）——图表所展示的，就是生产环境实际跑出的样子。

3. 增量的落地机制

3.1 IN/OUT/DEP 契约：D3 的机器可校验载体

以资金面库"隔夜资金价格分位"视角的模块契约为真实示例：

```
# data_contract/funding_pctile.yaml — 模块：资金面·隔夜价格分位
module: funding_pctile
version: 2.1

in:                                     # 输入声明：字段 + 数据源依赖
- dr001: {type: series, freq: day, aliases: [DR001, dr001_close]}
- dr007: {type: series, freq: day, aliases: [DR007, dr007_close]}
- start: {type: date}
- window: {type: int, default: 250}

out:                                    # 输出声明
- pctile: {type: scalar, unit: pct}
- regime: {type: label, enum: [宽松, 中性, 偏紧, 极紧]}

dep:                                    # 运行时依赖的模块
- scripts/funding_fetcher.py          # 取数引擎（自包含）
- knowledge/资金面.md                 # 阈值与回验基线
- config/funding.yaml                 # 参数与路由
```

示例要点：一条 DASH 对话里针对"隔夜资金分位"发起的调用，只加载 `funding_fetcher` 模块及其 `dep` 声明依赖，其余 33 个视角的代码不进上下文；外部调用方依据契约即可运行与复核，不必通读整套数据库。这正是 D3 的机器可校验性：字段是否齐备、类型是否匹配、缺数如何处理、依赖是否满足，均可逐项自动核查。

3.2 知识层回验基线：D4 的落地机制

回验基线的运行闭环：先校验框架（口径、数据链路、计算是否按契约执行），框架无误后才信任数值；若数值偏离基线，优先怀疑数据链路（缺数、错源、停更），而不是先改结论。更新方向由市场事件标注驱动：每次回验把"极端值"与事件日历对照，能解释则标注沉淀，不能解释则追查数据，中枢漂移后更新阈值。前述 90%→70% 案例正是这一闭环的一次完整运转。

3.3 五层中间件：模块粒度下的独立可替换

分层思想延续 v1 主张（数据源、取数、知识、计算、输出五层，详见 v1 论文）。Skill 2.0 的增量在于：分层与模块化叠加后，**每一层的可替换性在模块粒度上成立**——换数据源只改取数层的映射，换输出形态只换输出层模板，任何一层的模块级实现变更都不迫使其它层联动

修改。五层结构在附录中以 ASCII 图给出机制示意，其作用是把 D1~D6 的模块约束安置进一条可替换的流水线。

4. 真实案例与测量

4.1 DASH 只取一模块（D2 证据）

在 13 个主题数据库、约 500 个分析逻辑、多本生产手册（资金面、信用利差、可转债、财务分析、基金久期）的运行规模下，模块粒度的收益直接可见。以 DASH 会话中一次资金面取数为案例（fig1）：资金面库 34 个视角，本次只调用 `scripts/funding_fetcher.py` 模块，其余 33 个视角不加载。三重收益：上下文占用与加载时间按比例下降；调用方只依赖该模块契约，与其它视角演进解耦；取数引擎可被其它会话独立复用而不必“借走”整套库。同一模式适用于信用利差库（11 视角 × 5 源）与可转债库（22 视角）。

4.2 上下文注入的量级对比（D2 量化，量级估算）

把上述机制换成量化的量级感：按生产库的模块体量估算，全量注入 34 视角资金面库的等价上下文约在 **128k token 量级**；按路径只取一个视角模块后，注入量降至 **4k token 量级**——相差约 30 倍，与 fig7 实测的“100%→1/30”一致。**以上为量级估算**：精确值随模块粒度、知识文件大小与候选视角数量浮动，本文不将其声明为精确测量。

4.3 手册拆解即模块化实例（D6 证据）

手册拆解本身是模块化的一次真实落地：介绍手册与架构手册从同一体系分家，架构手册成为集成者可直接对照代码验收的中间件文档面。其图表全部来自 DASH 实录与生产手册实拍的高保真复刻，采用 fioutput Navy 体系（fig2/fig3/fig5）——读者看到的案例图，是生产环境实际跑出的样子，而非示意图。

4.4 复用与版本（D5 证据）

资金面分位模块在生产中被信用、宏观等多库引用，契约版本 2.1 一次升级即全量受益；视角级回退只涉及引用方，不牵动其它视角。`docs/skill-matrix.md` 作为持续运行的模块注册表，记录模块→库的引用关系，使“卖方建组件、买方装配”的分工在文档层面可查证（fig10）。

5. 讨论

5.1 与 v1 的关系：增量而非替代

Skill 2.0 不否定 v1：四层架构、数据源切换、四条设计原则仍然是体系的地基，本文不重复这些内容即是对 v1 有效性的背书。两者共享同一立场——研究能力皆可工程化、需要时再组装；增量在于把封装粒度、契约、激活、知识维护、复用与交付诸维度各推进一步。对读者而言，v1 回答“是什么、能不能”，本文回答“跑大之后如何变好”。

5.2 局限

- **契约一次性成本**：模块与 IN/OUT/DEP 契约的设计、维护是持续投入，新建领域需从头设计模块边界与契约。
- **回验覆盖依赖人工**：回验基线的覆盖深度依赖人工标注市场事件与积累，未纳入基线的视角保护有限。
- **测量精度有限**：上下文注入比的精确值随模块体量浮动，4.2 节的 128k→4k 为量级估算；自包含取数引擎带来一定依赖冗余，模块间数据一致性需要中间件层显式校验。
- **验证主场景单一**：全量验证仍局限于固收。

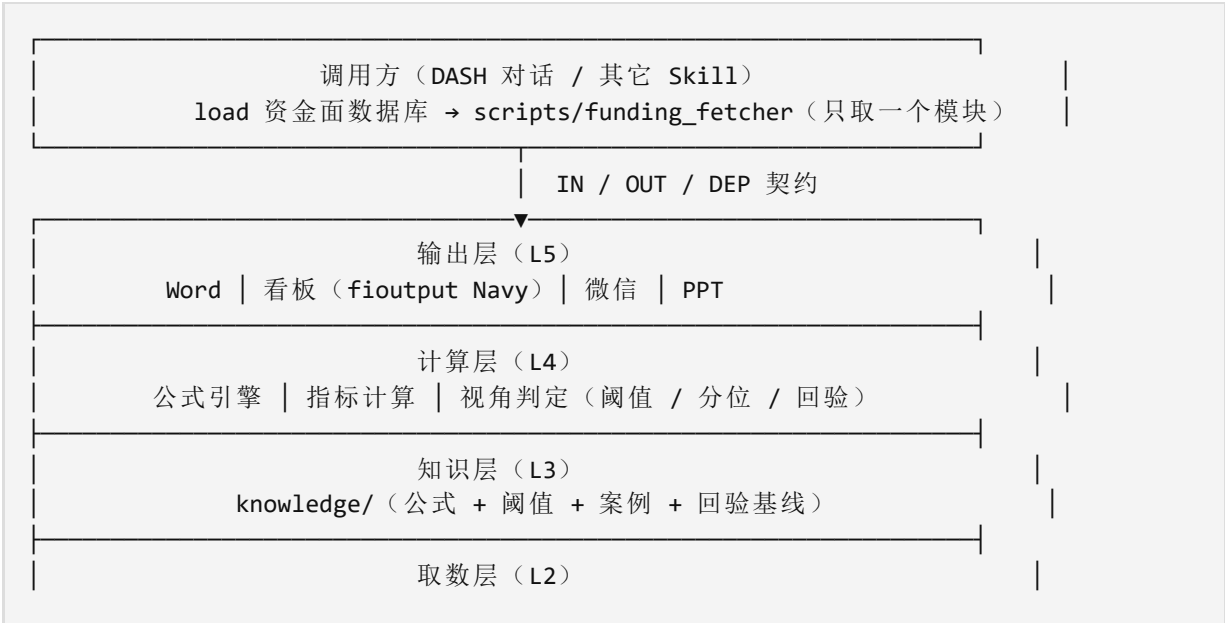
5.3 迁移展望

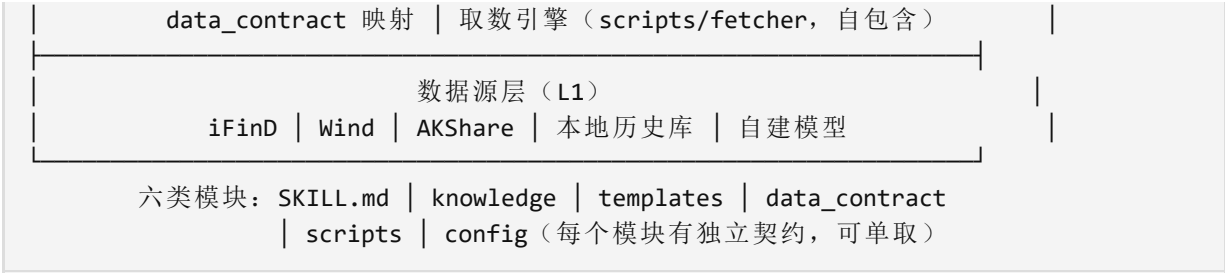
模块化抽象本身不绑定固收业务：取数、输出等层面跨领域基本可直接复用，主要新增在知识模块与计算模块。权益研究（行业数据库、选股因子库）、宏观研究（经济指标库、政策事件库）、量化策略（因子库、回测数据库）是自然候选。需要如实说明：**目前仅在固收跑通全量，权益、宏观、量化仍处于设计推演，迁移论据是抽象的通用性，而非完整实证。**

6. 结论

从 Skill 1.0 到 Skill 2.0，主线是六个增量：封装单位（D1）、激活与上下文（D2）、接口契约（D3）、知识基线（D4）、复用与维护（D5）、生产形态（D6）。每一维都把 Skill 从"整体可用的黑盒"推向"模块按需可用的中间件"，并各有生产证据支撑——34 视角只取一模块的约 1/30 上下文降幅、90%→70% 的回验纠错、手册拆解即模块化实例、模块级引用让一处维护多处受益。第二次工程化不是推翻第一次，而是把"研究能力可以封装"落实为"封装到模块、契约到接口、经验到基线"。

附录：Skill 2.0 模块化中间件架构全景图





持续更新。最新版本: github.com/timyefi/skill20-arch